

生物序列比對問題上考慮凸形間隔處罰函數的一些 $O(nm)$ 演算法

吳哲賢，王志倫

中華大學資訊工程研究所

300 新竹市東香 30 號

{jswu,m890202}@chu.edu.tw

摘要

假設 n, m 為兩序列長度, $n \geq m$ 。兩條 DNA 或胺基酸的生物序列比對問題上, 不考慮間隔處罰函數或只考慮線性間隔處罰函數, 其演算法的時間複雜度為 $O(nm)$; 考慮任意間隔處罰函數, 時間複雜度增為 $O(nm^2)$ 。若考慮生物上的意義, 凸形間隔處罰函數: $g(k)-g(k-1) \geq g(k+1)-g(k)$, 其中 k 為間隔長度, 是一個符合條件的良好選擇。針對凸形間隔處罰函數, Myers 及 Miller 於 1988 年首先提出時間複雜度為 $O(nm \log m)$ 的方法, 並找出一個特殊的凸形間隔處罰函數, 對數函數: $g(k)=A+B \log k$, 提出 $O(nm)$ 的演算法。本論文針對凸形間隔處罰函數, 首先推導出擁有 $O(nm)$ 演算法的凸形間隔處罰函數條件, 並提出一些符合條件的建議函數。

一、簡介

所謂 DNA 或胺基酸的生物序列比對(sequence alignment)問題, 是由二條有限長度的生物序列, 插入間隔形成長度相同的序列, 再根據評分矩陣及間隔處罰(gap penalty)函數, 找出分數最高的序列比對方式。當序列比對問題剛開始發展時, Needleman 及 Wunsch[5]於 1970 年, 利用動態規劃(dynamic programming)的方法, 提出不考慮間隔處罰函數的 $O(nm)$ 演算法, 其中 n, m 為序列長度, $n \geq m$ 。接著 Waterman、Smith 及 Beyer[7]於 1976 年, 提出考慮任意間隔處罰函數, 時間複雜度為 $O(nm^2)$ 的演算法。Gotoh[2]於 1982 年針對線性間隔處罰(affine gap penalty)函數: $g(k)=A+Bk$, A, B 為非負整數, k 為間隔長度; 提出時間複雜度是 $O(nm)$ 的演算法。

採用線性間隔處罰在生物序列比對上並非永遠正確, 特別是針對長間隔會有過重的處罰。Waterman[6]於 1984 年, 提出凸形間隔處罰(convex gap penalty)函數: $g(k)-g(k-1) \geq g(k+1)-g(k)$, 其中 k 為間隔長度; 它在長間隔發生頻率高的時候會比線性間隔處罰具有較高的適用性。他預測存在時間複雜度為 $O(nm \log m)$, 甚至 $O(nm)$ 的演算法。Myers 及 Miller[4]於 1988 年, 針對凸形間隔處罰函數, 使用候選者名單(candidate list)的方法, 首先提出時間複雜度為 $O(nm \log m)$ 的方法, 並找出一個特殊的凸形間隔

處罰函數, 對數(logarithmic)函數: $g(k)=A+B \log k$, 提出 $O(nm)$ 的演算法。爾後 Galil 及 Giancarlo[1]於 1989 年, Gusfield[3]於 1997 年, 也分別提出了時間複雜度一樣是 $O(nm \log m)$ 的改良演算法。

這篇論文是討論在序列比對中有關凸形間隔處罰上的問題。首先利用 Gusfield[3]於 1997 年設計的 $O(nm \log m)$ 演算法, 分析其特性, 並進一步推導出符合 $O(nm)$ 時間複雜度的凸形間隔處罰函數條件。序列比對工作並非毫無目的的使用任意間隔函數來進行運算, 接著找出一些既簡單又符合上述條件的凸形間隔處罰函數, 如此不但符合生物上的意義, 並且降低時間複雜度為 $O(nm)$ 。使得考慮凸形間隔處罰函數演算法的執行時間, 可以和不考慮間隔處罰函數演算法的執行時間, 具有一樣的時間複雜度。

二、考慮任意間隔處罰函數的 $O(nm^2)$ 演算法

假設 n, m 為兩序列長度, $n \geq m$ 。針對生物序列比對上, 考慮任意間隔處罰函數的問題, 首先定義一些變數,

$V(i, j)$: 第一序列前 i 個字母和第二序列前 j 個字母, 最高的序列比對分數

$G(i, j)$: 在 $V(i, j)$ 中, 第一序列第 i 個字母和第二序列第 j 個字母比對

$E(i, j)$: 在 $V(i, j)$ 中, 第一序列間隔和第二序列第 j 個字母比對

$F(i, j)$: 在 $V(i, j)$ 中, 第一序列第 i 個字母和第二序列間隔比對

$S(i, j)$: 第一序列第 i 個字母和第二序列第 j 個字母比對分數

$g(k)$: 間隔長度為 k 的比對分數

接著利用以下動態規程式,

$V(i, j) = \max[E(i, j), F(i, j), G(i, j)],$

$G(i, j) = V(i-1, j-1) + S(i, j),$

$E(i, j) = \max[V(i, k) - g(j-k)], 0 \leq k \leq j-1,$

$F(i, j) = \max[V(k, j) - g(i-k)], 0 \leq k \leq i-1,$

$V(i, 0) = -g(i),$

$V(0, j) = -g(j),$

$E(i, 0) = -g(i),$

$F(0, j) = -g(j).$

可以得到時間複雜度為 $O(nm^2)$ 的演算法。考慮 $E(i, j)$ 矩陣, 其中每一列需 $O(m^2)$ 的計算時間, 共有 n 個列, 所以共需 $O(nm^2)$ 時間, $F(i, j)$ 反之

亦同；而 $G(i,j)$ 只需 $O(nm)$ 時間。所以最後求 $V(m,n)$ 需要 $O(nm^2)$ 的時間複雜度。

三、考慮凸形間隔處罰函數的 $O(nm \log m)$ 演算法

針對凸形間隔處罰函數的問題，假設 n, m 為兩序列長度， $n \geq m$ 。固定 E, F, V 及 G 矩陣的每一列，考慮 E 矩陣某一固定列 (F 矩陣反之亦同)，

$$E(j) = \max[V(k) - g(j-k)], 0 \leq k \leq j-1,$$

為簡化起見，假設

$$C(k,j) = V(k) - g(j-k),$$

則

$$E(j) = \max[C(k,j)], 0 \leq k \leq j-1.$$

根據 Gusfield[3]於 1997 年提出的 $O(nm \log m)$ 演算法，針對每一列，利用反前進動態規劃 (reversed forward dynamic programming) 方法，演算法敘述如下：

Initialize the end-of-block list to contain the single number m .

Initialize the associated pointer list to contain the single number 0.

For $j:=1$ to m **do**

Begin

Set k to be the first pointer on the b -pointer list.

$$E(j) = C(k,j);$$

$$V(j) = \max[E(j), F(j), G(j)];$$

{As before we assume that the needed F and G values have been computed.}

{Now see how j 's candidates change the block-partition.}

Set j' equal to the first entry on the end-of-block list.

{look for the first index s in the end-of-block list where j loses}

If $C(b(j'), j+1) < C(j, j+1)$ **then** { j 's candidate wins one}

Begin

While

The end-of-block list is not empty and $C(b(j'), j') < C(j, j')$ **do**

Begin

Remove the first entry on the end-of-block list, and remove the corresponding b -pointer.

If the end-of-block list is not empty **then**
Set j' to the new first entry on the end-of-block list.

End;

End{while};

If the end-of-block list is empty **then**

Place m at the head of that list;

Else{when the end-of-block list is not empty}

Begin

Let p_s denote the first end-of-block empty.

Using binary search over the cells in block s , find the right-most point p in that block such that $C(j, p) > C(b_s, p)$.

Add p to the head of the end-of-block list;

End;

Add j to the head of the b pointer list.

End;

End.

上述演算法中，while loop 部份的執行時間，Gusfield[3]已證明整個列只需 $O(m)$ 的時間複雜度。利用二元搜尋法找 p 值需 $O(\log m)$ 的時間，總共 m 個搜尋便需要 $O(m \log m)$ 的時間。因此每一列需 $O(m) + O(m \log m) = O(m \log m)$ ，而全部 n 列共需要 $O(nm \log m)$ 的時間複雜度。

四、考慮凸形間隔處罰函數的 $O(nm)$ 演算法

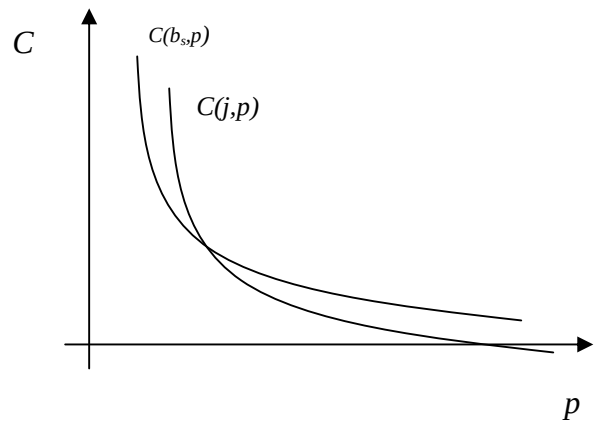
上節演算法中，搜尋 p 值的條件為：right-most point p in that block such that $C(j, p) > C(b_s, p)$ 。也就是說，找出最大的 p 整數值，滿足下列條件

$$C(j, p) > C(b_s, p),$$

即是

$$V(j) - g(p-j) > V(b_s) - g(p-b_s).$$

其中 g 為已知凸形間隔處罰函數， $j, b_s, V(j)$ 及 $V(b_s)$ 為已知數值，唯一未知數為 p 。函數圖形如下：



[定理一] 考慮凸形間隔處罰函數的序列比對問題，設計特定的凸形間隔處罰函數 g ，只要在常數時間可以解出最大的 p 整數值，滿足 $V(j) - g(p-j) > V(b_s) - g(p-b_s)$ ，其中 $j, b_s, V(j)$ 及 $V(b_s)$ 為已知數值，則存在 $O(nm)$ 的演算法。

證明：搜尋每一個 p 值，只需 $O(1)$ 的時間，每一列需搜尋 m 個 p 值，因此需 $O(m)$ 的時間。全部 n 列共需 $O(nm)$ 的時間複雜度，所以存在 $O(nm)$ 的演算法。

五、符合 $O(nm)$ 演算法條件的一些凸形間隔處罰函數

在此列舉一些符合 $O(nm)$ 的演算法的凸形間隔處罰函數。

【函數一】 $g(x) = \log x$

証明：Myers 及 Miller[4] 已於 1988 年，針對此一特殊的凸形間隔處罰函數：對數函數，提出 $O(nm)$ 的方法。現在推演如何在常數時間解出最大的 p 整數值。

$V(j) - \log(p-j) > V(b_s) - \log(p - b_s)$, $// j, b_s, V(j), V(b_s)$ 為已知數值

$\log(p - b_s) - \log(p-j) > V(b_s) - V(j)$, $// c1 = V(b_s) - V(j)$

$\log[(p - b_s)/(p-j)] > c1$, $// j > b_s, [(p - b_s)/(p-j)] > 1$

$(p - b_s)/(p-j) > 10^{c1}$, $// p > j$

$(j \cdot 10^{c1} - b_s)/(10^{c1} - 1) > p$, $// 10^{c1} - 1 > 0$

最後取符合上述不等式的最大的整數，即為 p 值。

【函數二】 $g(x) = a + ar + ar^2 + \dots + ar^{x-1} = [a(1 - r^x)]/(1 - r)$ ，其中 $a > 0, 1 > r > 0$ 。

証明：

$V(j) - [a(1 - r^{p-j})]/(1 - r) > V(b_s) - [a(1 - r^{p-b_s})]/(1 - r)$,

$[a(1 - r^{p-b_s})]/(1 - r) - [a(1 - r^{p-j})]/(1 - r) > V(b_s) - V(j)$,

$[a(r^{p-j} - r^{p-b_s})]/(1 - r) > c1$, $// c1 = V(b_s) - V(j)$

$[a \cdot r^{p-b_s}(r^{b_s-j} - 1)]/(1 - r) > c1$, $// j > b_s, 1 > r, r^{b_s-j} - 1 > 0$

$r^{p-b_s} > [c1 \cdot (1 - r)]/[a \cdot (r^{b_s-j} - 1)]$,

$p - b_s < \log_r c2$, $// 1 > r, c2 = [c1 \cdot (1 - r)]/[a \cdot (r^{b_s-j} - 1)]$

$p < b_s + \log_r c2$.

最後取符合上述不等式的最大整數，即為 p 值。

六、結論

針對凸形間隔處罰函數，本論文首先推導

出一個定理，說明擁有 $O(nm)$ 演算法的凸形間隔處罰函數條件。接著提出兩個計算簡單又符合條件的建議函數：對數函數， $g(x) = \log x$ ；以及等比級數和函數， $g(x) = a + ar + ar^2 + \dots + ar^{x-1} = [a(1 - r^x)]/(1 - r)$ ，其中 $a > 0, 1 > r > 0$ 。

針對任一凸形間隔處罰函數，找出 $O(nm)$ 演算法，依舊是一個開放未解的題目。涵蓋範圍擴大到任一間隔處罰函數，目前已知的演算法尚為 $O(nm^2)$ 的時間複雜度。如何改進執行時間，亦是一個值得研究的方向。

參考文獻

- [1] Z. Galil, R. Giancarlo, "Speeding up dynamic programming with applications to molecular biology", Theor. Comp. Sci., 64, p107-118, 1989.
- [2] O. Gotoh, "An improved algorithm for matching biological sequences", J. Mol. Boil., 162, p705-708, 1982.
- [3] D. Gusfield, "Algorithms on strings, Trees and sequences", Cambridge University Press, Cambridge, 1997.
- [4] W. Miller, E. W. Myers, "Sequence comparisons with concave weighting functions", Bull. Math. Biol., 50, p97-120, 1988.
- [5] S. B. Needleman, C. D. Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins", J. Mol. Boil., 48, p443-453, 1970.
- [6] M. S. Waterman, "Efficient sequence alignment algorithms", J. Theor. Biol., 108, p333-337, 1984.
- [7] M. S. Waterman, T. F. Smith, W. A. Beyer, "Some biological sequence metrics", Advances in Mathematics, 20, p367-387, 1976.